上海科技大学
ShanghaiTech University

CS240 Algorithm Design and Analysis

Midterm Recitation

CS240 TAs

Spring 2026

2026.04.24

Important Notes

- **April 28th from 13:00 PM to 14:40 PM.** Arrive 10 minutes early!
- **Classroom 201 in the Teaching Center.**
- **Covering:** Asymptotic Notation, Short Questions, Greedy Algorithm, Divide and Conquer, Dynamic Programming, Network Flow, NP
- **DO NOT CHEAT!**

Disclaimer

- Topics covered in this review may not appear in the exam.
- Topics not covered in this review may appear in the exam.

Outlines

- Asymptotic Notation
- Short Questions
- Greedy Algorithm
- Divide and Conquer
- Dynamic Programming
- Network
- NP

Asymptotic Notation

Asymptotic Order of Growth

- **Upper bounds.** T(n) is $O(f(n))$ if there exist constants c > 0 and n_0 >= 0 such that for all n >= n_0 we have T(n) <= c · f(n)
- **Lower bounds.** T(n) is $\Omega(f(n))$ if there exist constants c > 0 and n_0 >= 0 such that for all n >= n_0 we have T(n) >= c · f(n)
- **Tight bounds.** T(n) is $\Theta(f(n))$ if T(n) is both $O(f(n))$ and $\Omega(f(n))$.
  - Exist constants c_1, c_2, n_0, such that $c_1 f(n) <= T(n) <= c_2 f(n)$ for all n >= n_0
- Let $T(n) = \frac{1}{2}n^2 + 3n$. Which of the following statements are true?
  - $T(n) = O(n)$
  - $T(n) = \Omega(n)$
  - $T(n) = \Theta(n)$
  - $T(n) = O(n^3)$

$$f(n) = \quad \mathbf{O}(g(n)) \quad \Omega(g(n)) \quad \mathbf{o}(g(n)) \quad \Theta(g(n)) \quad \omega(g(n))$$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \quad 0 \qquad 0 < c < \infty \qquad \infty$$

Greedy Algorithm

Greedy Algorithm Review

Core idea

Build the solution step by step and make the best local choice at each stage.

A greedy algorithm never revisits earlier decisions, so the local rule must be safe — not just intuitive.

When greedy is correct, it is usually simpler and faster than dynamic programming or exhaustive search.

When it works

Greedy-choice property: an optimal solution can start with the greedy choice.

Optimal substructure: after one choice, the remaining subproblem is still optimizable.

Typical cost: O(n log n) after sorting or maintaining a priority queue.

General framework

1. Order the candidates

Sort intervals, deadlines, edges, or pages by a local priority.

2. Test feasibility

Only keep a choice if it stays compatible with the partial solution.

3. Update a small state

Examples: last finish time, current connected components, cache contents.

4. Prove correctness

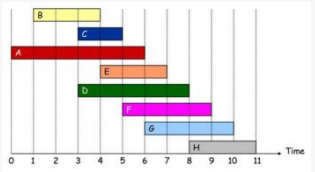
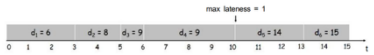
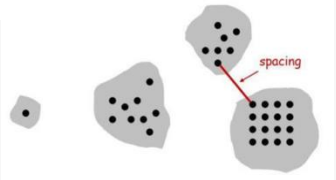
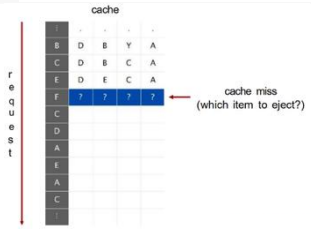
Use a proof template such as stays-ahead, exchange, or a structural property.

Review focus: local rule + feasibility check + correctness proof

Core Greedy Cases

上海科技大学 ShanghaiTech University

Each problem has a different objective, so the correct greedy rule is different as well.

Problem	Figure	Optimization goal	Greedy choice	Key intuition / course link
Interval Scheduling		Maximize the number of compatible jobs	Pick the compatible job with the earliest finish time	Finishing earlier leaves the most room for the rest of the schedule.
Minimizing Lateness		Minimize the maximum lateness	Schedule jobs by earliest deadline first	Urgent jobs should be protected first; short processing time alone is not enough.
Single-link k-clustering		Maximize spacing between clusters	Repeatedly merge the closest pair of clusters	This is exactly Kruskal's algorithm, stopped when k components remain.
Optimal Offline Caching		Minimize misses / evictions	Evict the item requested farthest in the future	If one item is needed last, removing it now is the safest choice.

Key lesson: a greedy strategy is not generic — it must match the exact objective and be justified by proof.

Greedy Proof Toolkit

Greedy proofs usually show that the local choice is safe — not just that it looks reasonable.

1. Greedy stays ahead

Compare the greedy partial solution with any other solution after each step.

Show the greedy prefix is never worse, so the final answer is optimal.

Typical link: interval scheduling.

2. Exchange argument

Start from an optimal solution and transform it toward the greedy one.

Each swap keeps the solution feasible and no worse than before.

Typical link: minimizing lateness; also useful in caching proofs.

3. Structural property

Prove each greedy move is forced by a theorem or invariant.

Examples: cut/cycle properties for MST; clustering through the MST structure.

Useful when the problem has a strong graph or matroid-like structure.

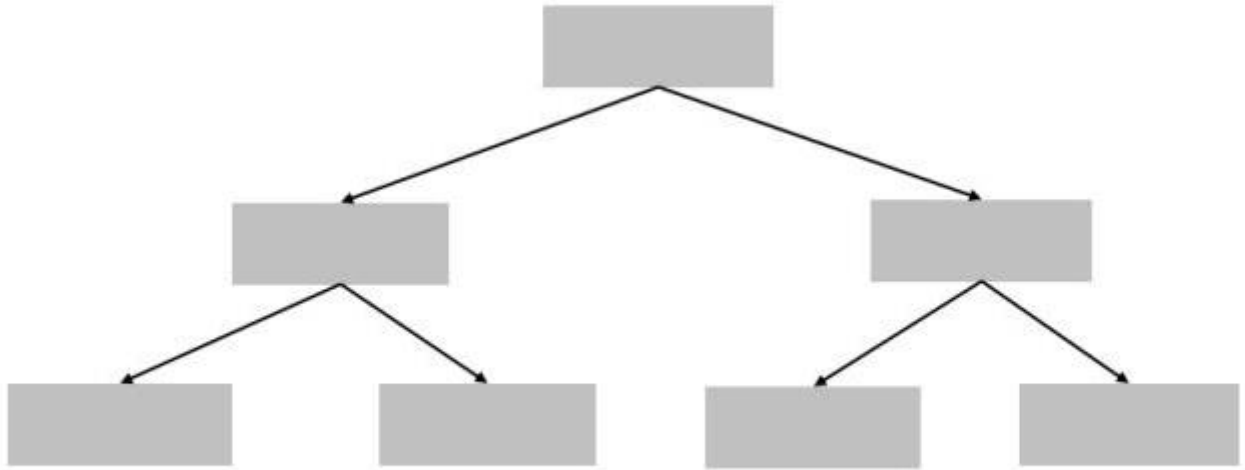
Fast exam checklist:

objective → local rule → feasibility condition → counterexample to tempting wrong rules → proof template → complexity

Divide-and-conquer

Divide-and-conquer

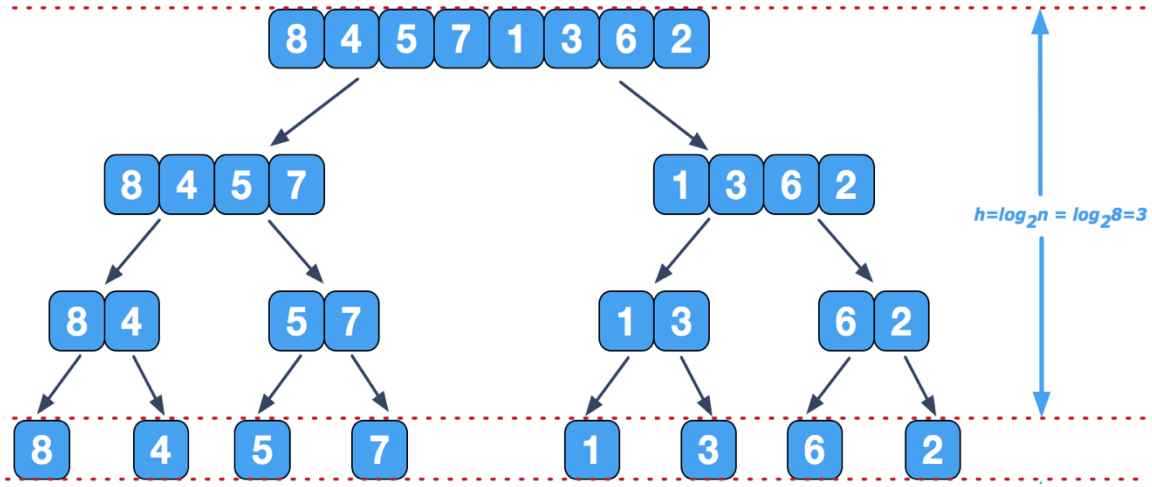
- **Greed.** Build up a solution incrementally, myopically optimizing some local criterion
- **Divide-and-conquer.** Break up a problem into a few sub-problems, solve each sub-problem independently and recursively, and combine solution to sub-problems to form solution to original problem



Mergesort

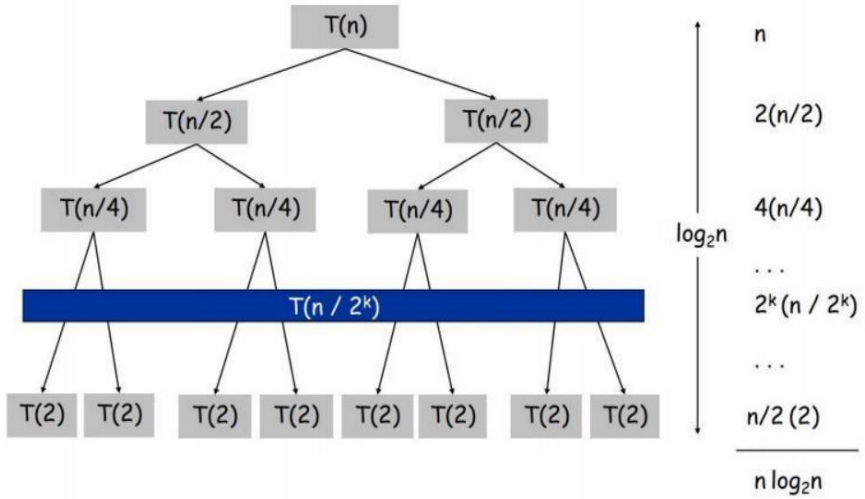
Mergesort

- Divide array into two halves
- Recursively sort each half
- Merge two halves to make sorted whole



Proof by Recursive Tree

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$



Proof by Induction

- Base case: n = 1
- Inductive hypothesis: $T(n) = n\log_2 n$
- Goal: show that $T(2n) = 2n \log_2(2n)$

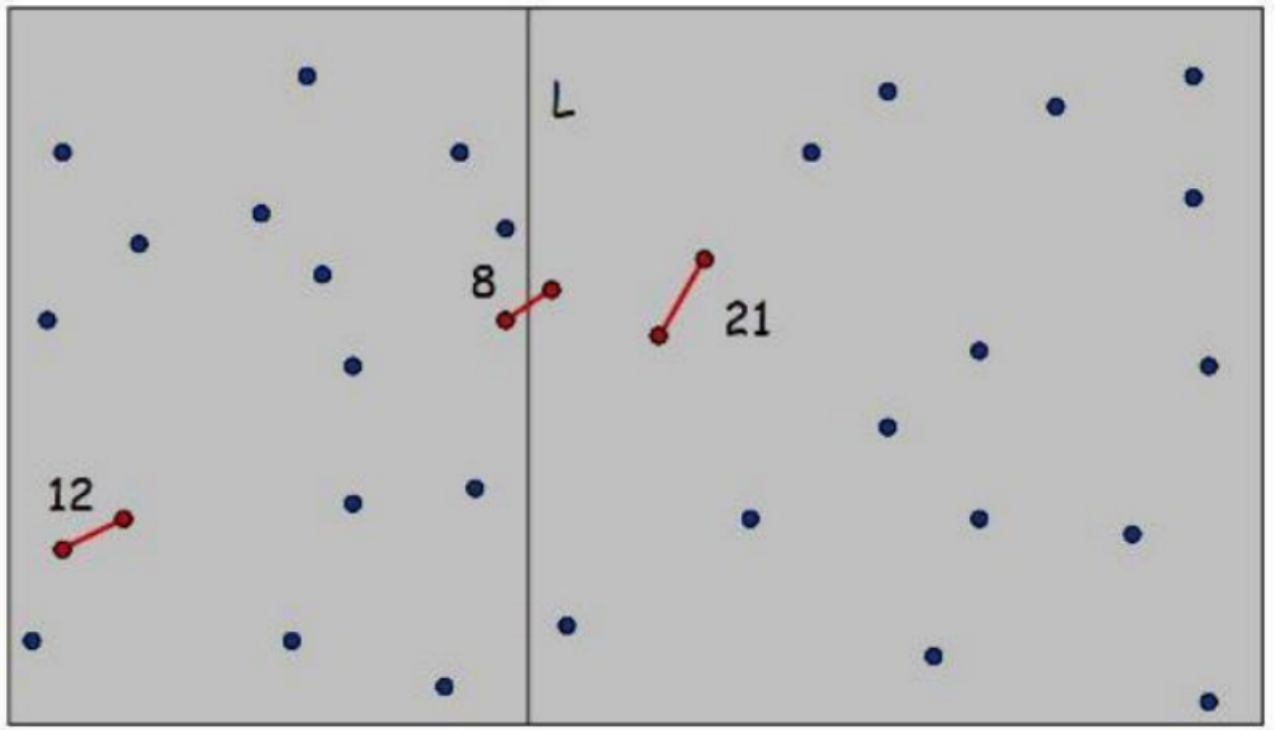
$$\begin{aligned} T(2n) &= 2T(n) + 2n \\ &= 2n\log_2 n + 2n \\ &= 2n(\log_2(2n) - 1) + 2n \\ &= 2n\log_2(2n) \end{aligned}$$

Closest Pair of Points

- **Closest Pair** Given n points in the plane, find a pair with smallest Euclidean distance between them

Algorithm

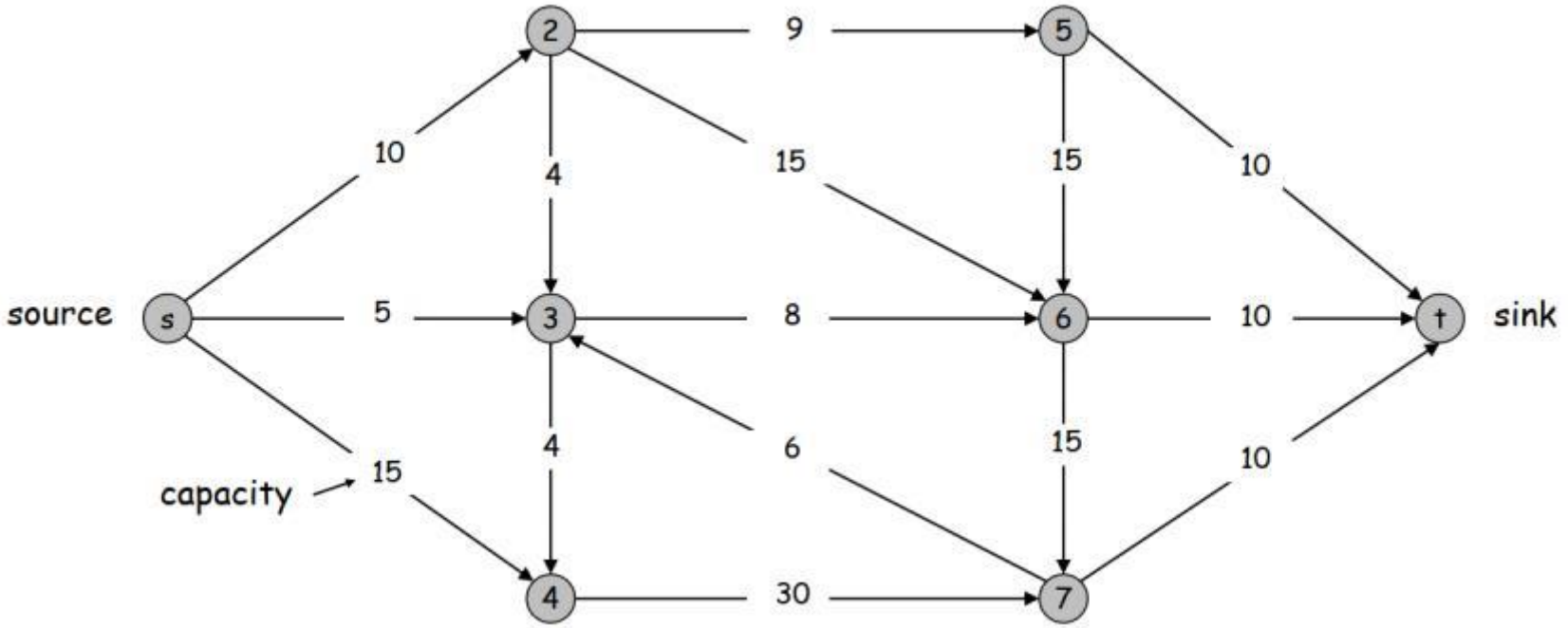
- Divide: draw vertical line L so that roughly ½n points on each side
- Conquer: find closest pair in each side recursively
- **Combine:** find closest pair with one point in each side ← $\Theta(n^2)$
- Return best of 3 solutions



Network Flow

Network Flow

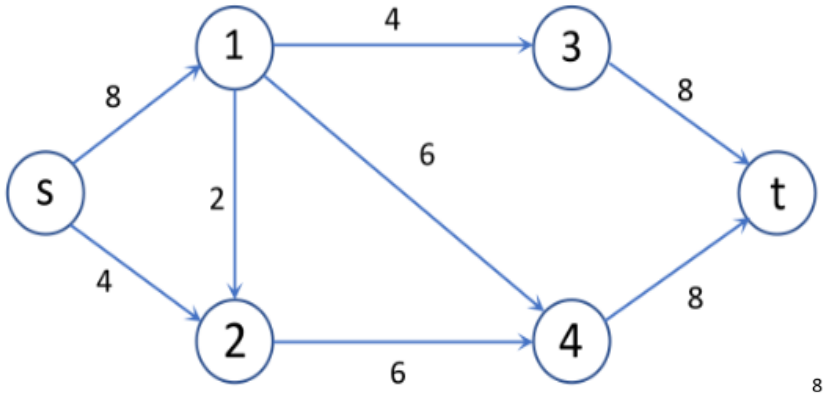
- **Flow network**
  - Abstraction for material **flowing** through the edges
  - G = (V, E) = directed graph
  - Two distinguished nodes: s = source, t = sink
  - c(e) = nonnegative capacity of edge e



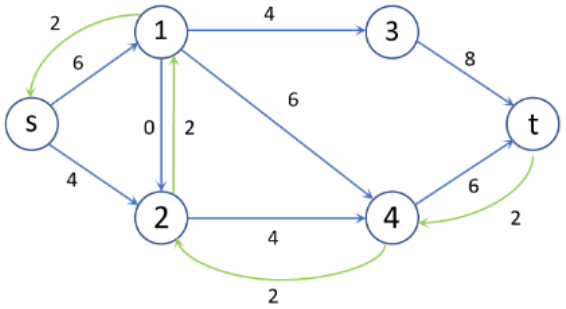
Ford-Fulkerson Algorithm

- **Ford-Fulkerson Algorithm**
  - Start with f(e) = 0 for all edge e ε E
  - Find an augmenting path P in the residual graph G_f
  - Augment flow along path P
  - Repeat until you get stuck

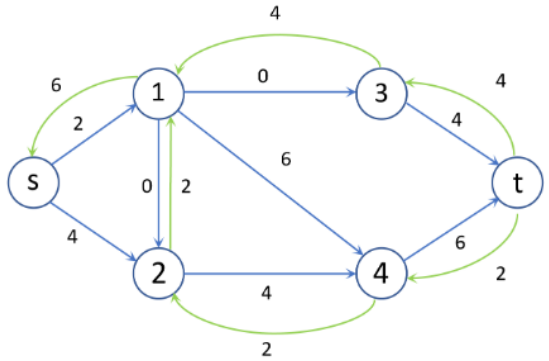
HW2 Problem4: During each iteration, choose the lexicographically smallest available augmenting path.



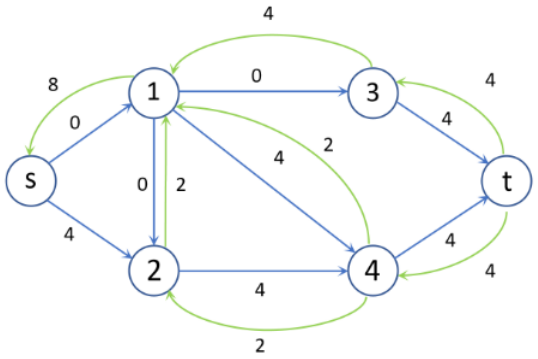
s → 1 → 2 → 4 → t



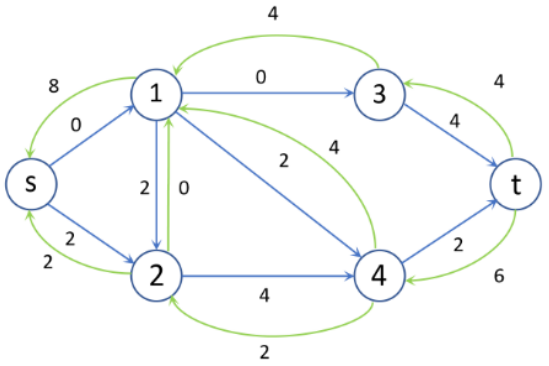
s → 1 → 3 → t



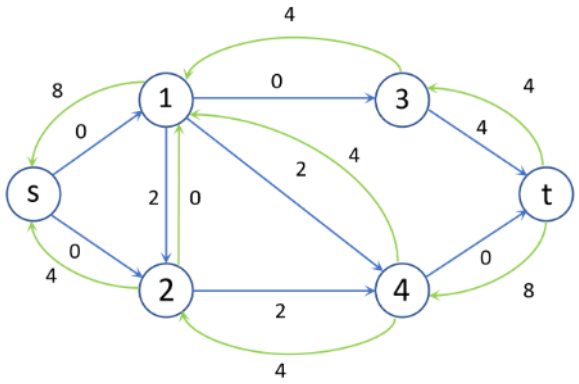
s → 1 → 4 → t



s → 2 → 1 → 4 → t



s → 2 → 4 → t



Max-flow and Min-Cut

- **Augmentingpath theorem.** Flow f is a max flow if there are no augmenting paths
- **Max-flow min-cut theorem.** The value of the max flow is equal to the value of the min cut
- **Proof strategy.** We prove both simultaneously by showing the equivalence of the following three conditions for any flow f:

  (i) There exists a cut (A, B) such that v(f) = cap(A, B)

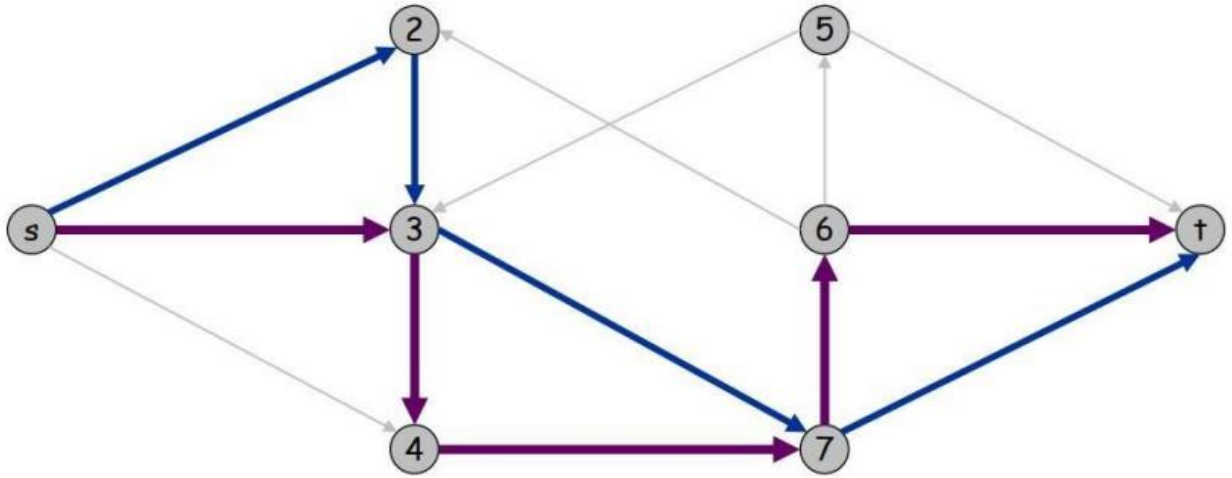
  (ii) Flow f is a max flow

  (iii) There is no augmenting path relative to f

Edge Disjoint Paths

- **Disjoint path problem.** Given a digraph G = (V, E) and two nodes s and t, find the max number of edge-disjoint s-t paths

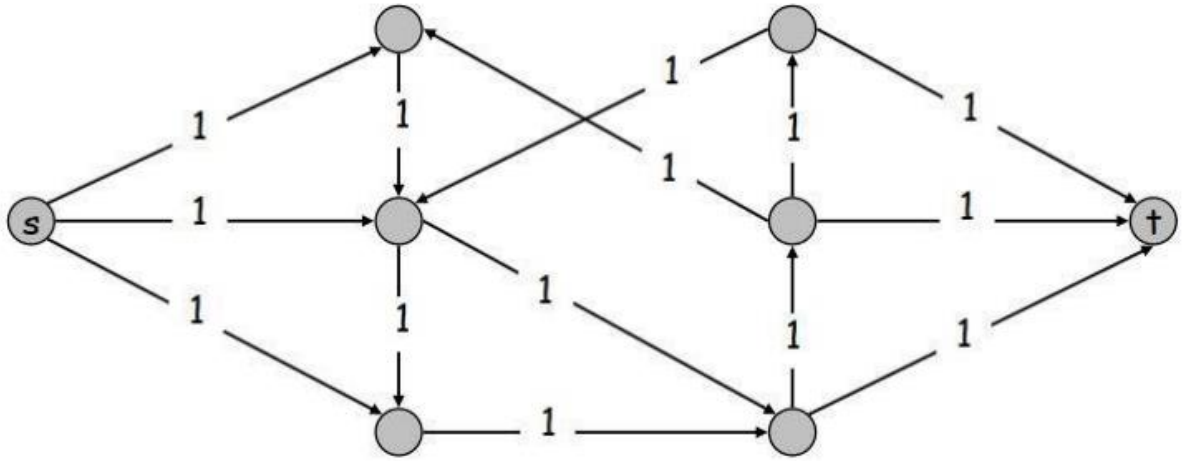
- **Def.** Two paths are edge-disjoint if they have no edge in common



上海科技大学
ShanghaiTech University
立志成才 报国裕民

Edge Disjoint Paths

- **Theorem.** Max number edge-disjoint s-t paths equals max flow value



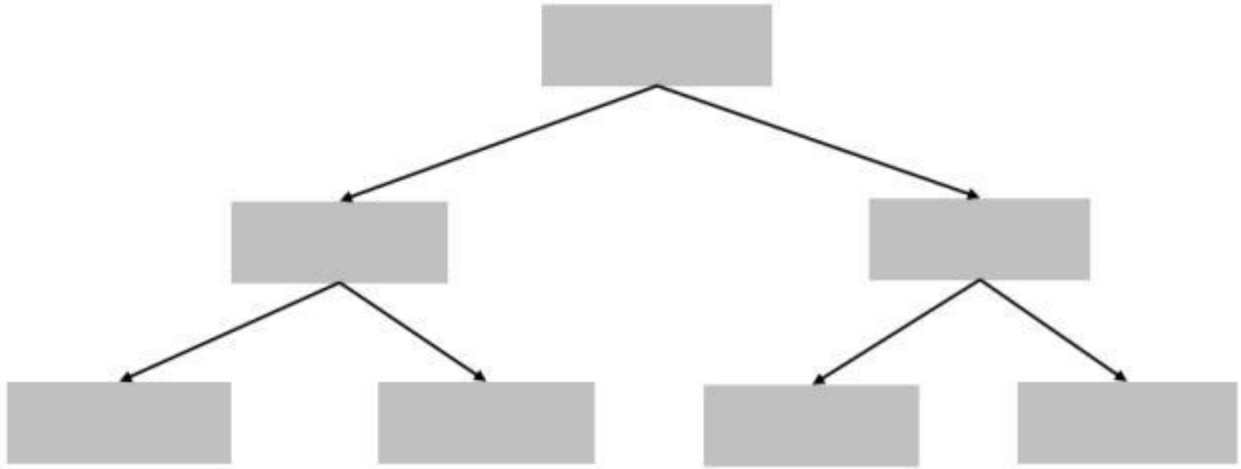
Pf. <= $\boxed{\text{max edge-disjoint paths} \leq \text{max flow}}$

- Suppose there are k edge-disjoint paths $P_1, \ldots, P_k$
- Set f(e) = 1 if e participates in some path P_i; else set f(e) = 0
- Since paths are edge-disjoint, f is a flow of value k

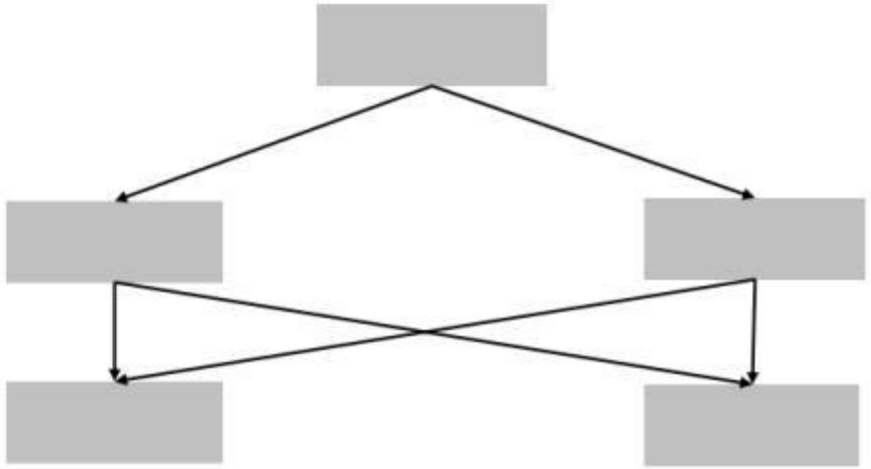
Dynamic Programming

Divide-and-conquer VS. Dynamic Programming

Divide-and-conquer



Dynamic programming



Dynamic Programming

- **Basic idea**
  - Polynomial number of sub-problems with a natural ordering from smallest to largest
  - Optimal solution to a sub-problem can be constructed from optimal solutions of smaller sub-problems
  - Sub-problems are overlapping!

- **Guideline**
  - Define the sub-problems
    - OPT(…)
  - Write down the recursive formulas
    - Ex: OPT(i) = max(f(OPT(j)), g(OPT(k)), …), j, k < i
  - Compute the formulas either bottom-up or top-down

Dynamic Programming

- **Algorithms**
  - Weighted interval scheduling
    - 1D array; binary choice
  - Knapsack
    - 2D array; adding a new variable (weight limit)
  - RNA secondary structure
    - 2D array: intervals
  - Sequence Alignment
    - 2D array: prefix alignment
  - Sequence Alignment in Linear Space
    - Combination of divide-and-conquer and dynamic programming
  - Shortest path with negative edges
    - (Bellman-Ford) 2D array: shortest path with edge number <= i
  - Distance Vector Protocol
  - Negative Cycle Detection

Dynamic Programming

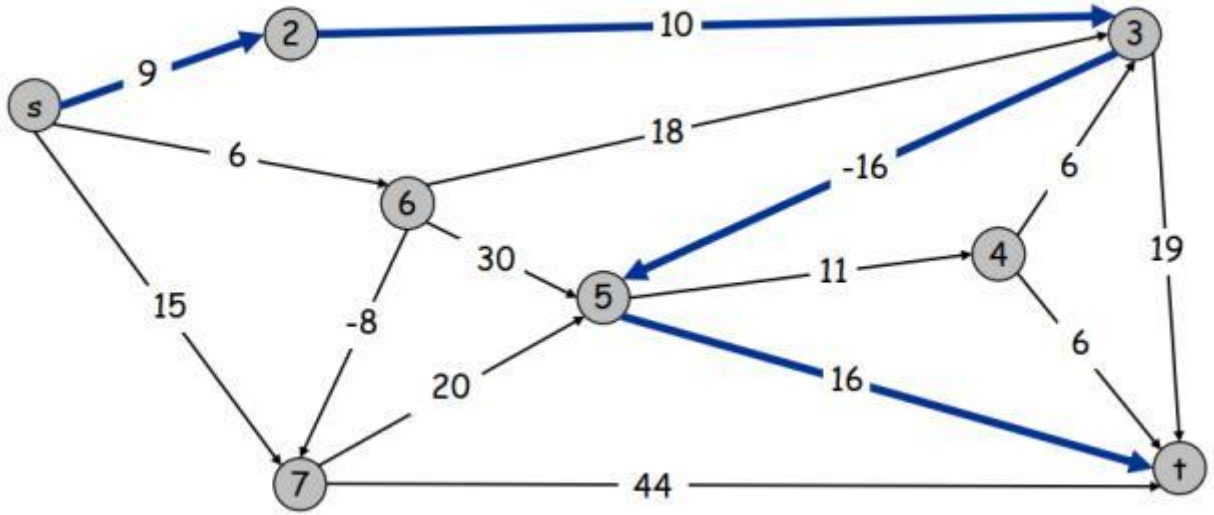
--- Shortest paths

Shortest Paths

- Shortest path problem. Given a directed graph G = (V, E), with edge weights c_{vw}, find shortest path from node s to node t

allow negative weights

- Ex. Nodes represent agents in a financial setting and c_{vw} is cost of transaction in which we buy from agent v and sell immediately to w



Shortest Paths: Failed Attempts

- **Dijkstra's algorithm**

Steps:

1. Initialize $dist[s] = 0$, others $= \infty$
2. Use a **priority queue**
3. Repeatedly extract node with minimum distance
4. Relax all outgoing edges

Time Complexity:

- $O((V + E) \log V)$ with heap

Requirement:

- ✅ Edge weights must be **non-negative**

- **Bellman-Ford algorithm**

Steps:

- Relax all edges $V - 1$ times
- One more pass to detect negative cycles

Time Complexity:

- $O(VE)$

Capability:

- ✅ Handles **negative weights**
- ✅ Detects **negative cycles**

NP and Computational Intractability

Key Concepts

- **Decision problem**
  - Answer yes/no

- **P.** Decision problems for which there is a **poly-time** algorithm
- **NP.** Decision problems for which there exists a **poly-time** certifier
  - Algorithm C(s, t) is a **certifier** for problem X if for every string s, $s \in X$ iff there exists a string t such that C(s, t) = yes

- **Co-NP.** Complements of decision problems in NP
- **EXP.** Decision problems for which there is an **exponential-time** algorithm
- **Claim.** $P \subseteq NP$, co-$NP \subseteq EXP$

Certifiers and Certificates: 3-Satisfiability

- **SAT**. Given a CNF formula ϕ, is there a satisfying assignment?
- **Certificate**. An assignment of truth values to the n Boolean variables
- **Certifier**. Check that each clause in ϕ has at least one true literal

- **Ex.**

$$(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (x_1 \vee x_2 \vee x_4) \wedge (\overline{x_1} \vee \overline{x_3} \vee \overline{x_4})$$

instance s

$$x_1 = 1,\ x_2 = 1,\ x_3 = 0,\ x_4 = 1$$

certificate t

- **Conclusion**. SAT is in NP

Polynomial-Time Reduction

- **Reduction.** Problem X **polynomial-time reduces** to problem Y if arbitrary instances of problem X can be solved using:
  - Polynomial number of standard computational steps, plus
  - Polynomial number of calls to oracle that solves problem Y

- **Notation.** $X \leq_p Y$

- **Common approach: polynomial transformation**
  - Given any input x to X, **construct** an input y in poly-time such that x is a yes instance of X **iff** y is a yes instance of Y

NP-Completeness & NP-Hard

- **NP-hard.** [Bell Labs, Steve Cook, Ron Rivest, Sartaj Sahni] A problem such that every problem in NP reduces to it

$$\forall X \in NP,\ X \leq_p Y$$

- **NP-complete.** A problem in NP such that every problem in NP polynomial reduces to it

$$NP - complete = NP \cap \text{NP-hard}$$

- **Remark.** Once we establish first "natural" NP-complete problem, others fall like dominoes
- **Recipe to establish NP-completeness of problem Y**
  - Step 1. Show that Y is in NP
  - Step 2. Choose an NP-complete problem X
  - Step 3. Prove that X ≤p Y

- **Theorem.** If X is NP-complete problem and Y is a problem in NP with the property that X ≤p Y, then Y is NP- complete
- **Pf.** Let W be any problem in NP. Then W ≤p X ≤p Y
  - By transitivity, W ≤p Y
  - Hence Y is NP-complete

By definition of NP-complete

By assumption

Review of 3-SAT

Review of 3-SAT

- **Literal:** A Boolean variable or its negation x_i or $\overline{x_i}$
- **Clause:** A disjunction of literals
- **Conjunctive normal form:** A propositional formula **Φ** that is the conjunction of clauses $C_j = x_1 \vee \overline{x_2} \vee x_3$
- **SAT.** Given CNF formula **Φ**, does it have a satisfying truth assignment? $\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$
- **3-SAT:** SAT where each clause contains exactly 3 literals

Ex: $(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$

Yes: x_1 = true, x_2 = true, x_3 = false, x_4 = false.

NP-Completeness

- **Observation.** All problems below are NP-complete and polynomial reduce to one another!